

# AI ASSESSED CODING

## Assignment-13.3

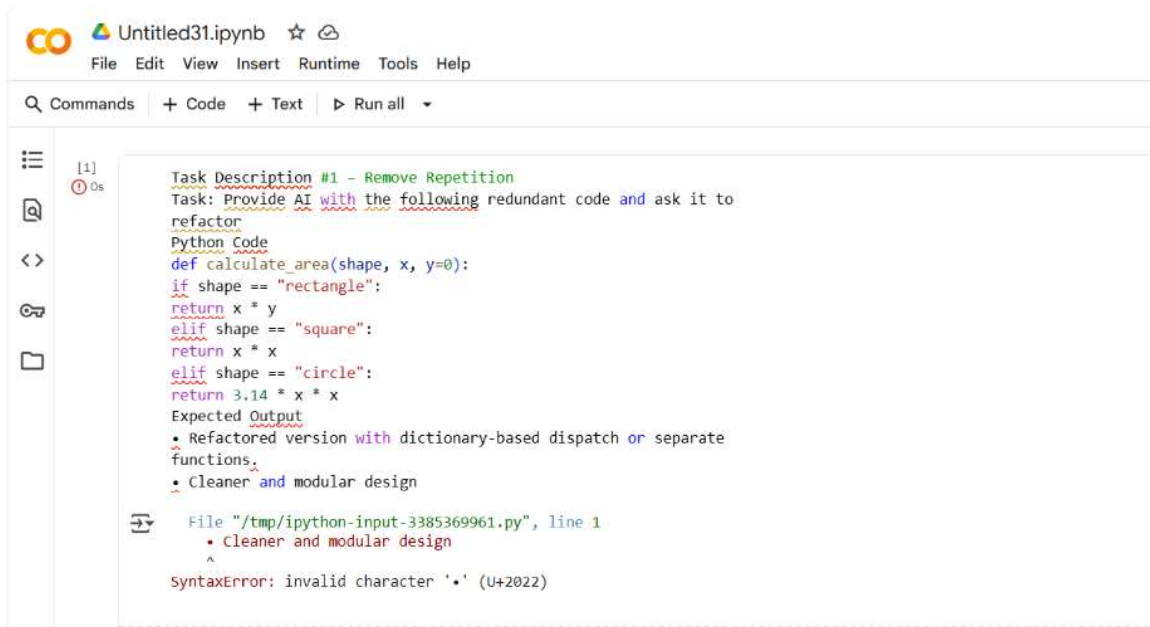
NAME:-B.PRATHAMKOUSHIK

ROLLNO:-2403A52182

1)

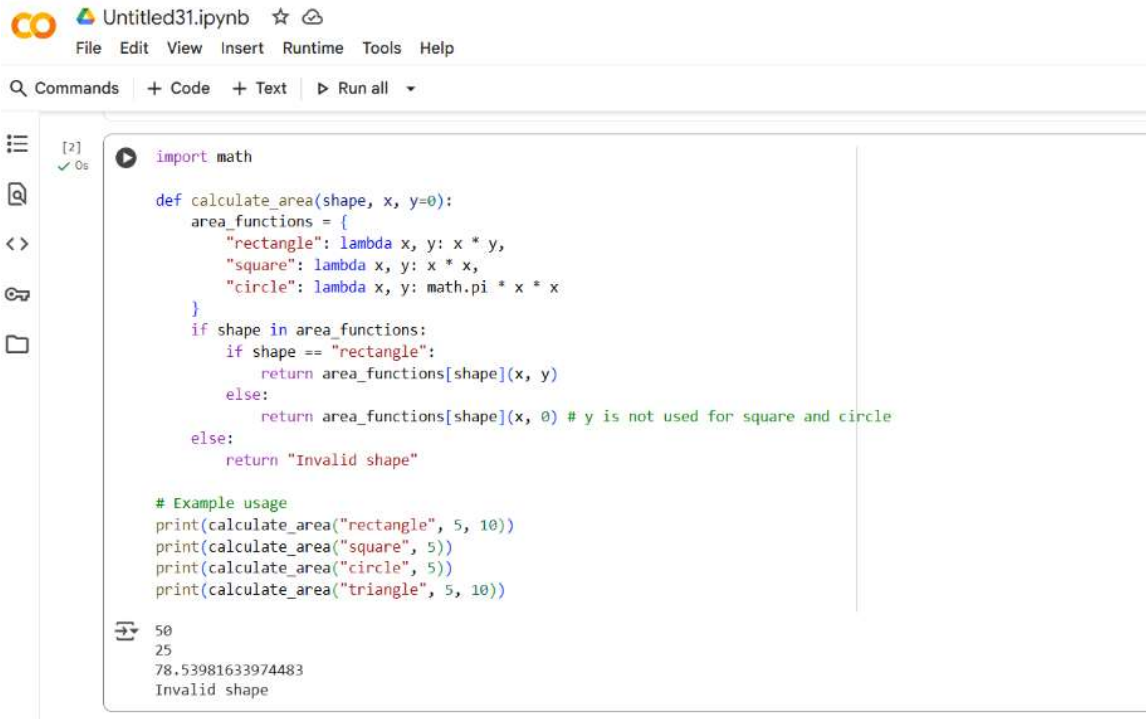
def calculate\_area(shape, x, y=0): if shape == "rectangle": return x \* y elif shape == "square": return x \* x elif shape == "circle": return 3.14 \* x \* x

refactor this code



The screenshot shows a Jupyter Notebook interface with the following content:

- Task Description #1 - Remove Repetition**
- Task:** Provide AI with the following redundant code and ask it to refactor
- Python Code**
- ```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```
- Expected Output**
- Refactored version with dictionary-based dispatch or separate functions.
  - Cleaner and modular design
- File "/tmp/ipynb-input-3385369961.py", line 1**
- Cleaner and modular design
- SyntaxError: invalid character '.' (U+2022)**



The image shows a Jupyter Notebook interface with a file named 'Untitled31.ipynb'. The notebook contains a Python script that defines a function 'calculate\_area' which takes a shape and dimensions as input and returns the area. The script includes example usage and handles an invalid shape by returning 'Invalid shape'.

```
import math

def calculate_area(shape, x, y=0):
    area_functions = {
        "rectangle": lambda x, y: x * y,
        "square": lambda x, y: x * x,
        "circle": lambda x, y: math.pi * x * x
    }
    if shape in area_functions:
        if shape == "rectangle":
            return area_functions[shape](x, y)
        else:
            return area_functions[shape](x, 0) # y is not used for square and circle
    else:
        return "Invalid shape"

# Example usage
print(calculate_area("rectangle", 5, 10))
print(calculate_area("square", 5))
print(calculate_area("circle", 5))
print(calculate_area("triangle", 5, 10))
```

The output of the script is displayed below the code cell:

```
50
25
78.53981633974483
Invalid shape
```

2)

Task Description #2 – Error Handling in Legacy Code Task: Legacy code without proper error handling Python Code

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()
    f.close()
    return data
```

Expected refactors with `open()` and `try-except`:


Untitled31.ipynb
☆
🔗

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all ▼

☰

🔍

⏪

🔑

📁

[3]

🔴 Os

Task Description #2 - Error Handling in Legacy Code

Task: Legacy function without proper error handling

Python Code

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()
    f.close()
    return data
```

Expected Output:

AI refactors with with open() and try-except:

```
File "/tmp/ipython-input-3249895895.py", line 1
Task Description #2 - Error Handling in Legacy Code
^
SyntaxError: invalid syntax
```

Next steps: [Explain error](#)

[4]

🟢 Os

Start coding or generate with AI.

[5]

🟢 Os

▶ `import os`

```
def read_file(filename):
    try:
        with open(filename, "r") as f:
            data = f.read()
        return data
    except FileNotFoundError:
        except FileNotFoundError:
```

▶ `import os`

```
def read_file(filename):
    try:
        with open(filename, "r") as f:
            data = f.read()
        return data
    except FileNotFoundError:
        return f"Error: The file '{filename}' was not found."
    except Exception as e:
        return f"An error occurred: {e}"

# Example usage (assuming a file named 'my_file.txt' exists)
# Create a dummy file for testing
with open('my_file.txt', 'w') as f:
    f.write("This is a test file.")

print(read_file('my_file.txt'))
print(read_file('non_existent_file.txt'))

# Clean up the dummy file
os.remove('my_file.txt')
```

🔗

This is a test file.  
Error: The file 'non\_existent\_file.txt' was not found.

3)

Task: Provide this legacy class to AI for readability and improvements: Python Code class Student: def \_\_init\_\_(self, name, age, mark1, mark2, mark3): self.n = name self.a = age self.m1 = mark1 self.m2 = mark2 self.m3 = mark3 def details(self): print("Name:", self.n, "Age:", self.a) def total(self): return self.m1+self.m2+self.m3 Expected Output: • AI improvements (adds docstrings, improves readability, adds sum(self.marks) if marks stored in a list)

```
class Student:
    """Represents a student with name, age, and marks in three subjects."""
    def __init__(self, name, age, mark1, mark2, mark3):
        """
        Initializes a Student object.

        Args:
            name (str): The name of the student.
            age (int): The age of the student.
            mark1 (int): The mark in the first subject.
            mark2 (int): The mark in the second subject.
            mark3 (int): The mark in the third subject.
        """
        self.name = name
        self.age = age
        self.mark1 = mark1
        self.mark2 = mark2
        self.mark3 = mark3

    def details(self):
        """Prints the name and age of the student."""
        print(f"Name: {self.name}, Age: {self.age}")

    def total(self):
        """Calculates and returns the total marks of the student."""
        return self.mark1 + self.mark2 + self.mark3

# Example usage
student1 = Student("Alice", 16, 85, 90, 78)
student1.details()

# Example usage
student1 = Student("Alice", 16, 85, 90, 78)
student1.details()
print(f"Total Marks: {student1.total()}")
```

 Name: Alice, Age: 16  
Total Marks: 253

4)

Task Description #4 – Inefficient Loop Refactoring Task: Refactor inefficient loop with AI help Python Code `nums = [1,2,3,4,5,6,7,8,9,10]`  
`squares = []` for i in nums: `squares.append(i * i)` Expected Output: AI suggested a list comprehension

Task Description #4 – Inefficient Loop Refactoring

Task: Refactor this inefficient loop with AI help

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```

Expected Output: AI suggested a list comprehension



File "/tmp/ipython-input-3519143920.py", line 1

Task Description #4 – Inefficient Loop Refactoring

^

SyntaxError: invalid syntax

Next steps:

[Explain error](#)

Start coding or [generate](#) with AI.



```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
# Refactored using a list comprehension
```

```
squares = [i * i for i in nums]
```

```
print(squares)
```



```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```